

README

HelixCode — docs/qa/ End-User Evidence Tree

Revision: 3 Last modified: 2026-06-22T00:00:00Z

Field	Value
Revision	3
Created	2026-05-28
Last modified	2026-06-22
Status	active (convention established — BLOCKING release gate enforced)
Status summary	Establishes the §11.4.83 per-feature end-user evidence convention + seed worked example (HXC-016). <code>scripts/verify_qa_evidence.sh</code> runs ADVISORY (warn-mode) by default and ENFORCING (blocking, exit 1) under <code>--enforce --since <baseline></code> ; the operator authorised promotion to a blocking release gate on 2026-05-28 (HXC-019), wired into <code>scripts/release-gate-test.sh</code> via <code>scripts/gates/qa_evidence_gate.sh</code> . Release-gate ONLY — not wired into pre-commit / pre-push hooks.
Authority	constitution submodule <code>Constitution.md §11.4.83 (docs/qa/end-user evidence mandate)</code>

Table of contents

- [What this tree is](#)
- [Run-id format](#)
- [What a transcript must contain](#)
- [Materials are committed in-repo](#)
- [Bot / agent-driven QA](#)
- [Directory layout](#)
- [Enforcement gate](#)
- [Worked example](#)

What this tree is

Per constitution submodule `Constitution.md` §11.4.83 (`docs/qa/` end-user evidence mandate), **every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials** (screenshots, request/response payloads, audio, file uploads) committed under `docs/qa/<run-id>/` — one directory per feature run.

The forensic anchor (verbatim 2026-05-22 operator mandate, quoted from §11.4.83):

“every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

A feature with no QA transcript is itself a §11.4 / §107 PASS-bluff: it claims to work but carries no auditable runtime evidence that an end user actually exercised it through the same interface they will use in production.

Run-id format

`<run-id>` MUST be monotonic + greppable. Use one of:

- **The workable-item ticket id** — HelixCode’s per-project prefix convention (`docs/Issues.md` “Prefix convention” section). For meta-repo / project-wide / governance / infrastructure work the prefix is HXC (e.g. HXC-016, HXC-019); submodule-scoped work uses its own prefix (HXA, etc.). This is the **preferred** form because it binds the QA evidence to the tracker entry in `docs/Issues.md` / `docs/Fixed.md`.
- **An ISO-8601 UTC timestamp** — YYYY-MM-DDTHHMMSSZ (e.g. 2026-05-28T143000Z) when a run is not tied to a single ticket.

Once chosen, a `<run-id>` directory is append-only — never renamed, never reused for a different feature.

What a transcript must contain

Per §11.4.83 operative rule (2):

1. **Full bidirectional thread.** Every prompt/command sent **and** every response received **and** every error message **and** every state change observed. A one-sided record (“we sent X”) is NOT a transcript; both halves are required.
2. The interface exercised MUST be the **same interface the end user will use in production** (CLI invocation, HTTP request, GUI action, etc.) — not an internal shortcut that bypasses the user-visible path.
3. Where the feature produces sink-side / downstream output (audio, video, network, rendered UI), the transcript MUST cite positive sink-side captured evidence per §11.4.68 / §11.4.69 — never config-only, never metadata-only.

The canonical transcript file is `docs/qa/<run-id>/transcript.md`.

Materials are committed in-repo

Per §11.4.83 operative rule (3): attached materials **MUST** be committed **alongside the transcript, in-repo** — screenshots in `.png`, payloads in `.json / .txt`, audio in `.wav / .mp3`, etc. External-only links (Slack URL, Drive URL) are a §11.4.13 sink-side violation; the evidence **MUST** live in the repository so the audit replay works offline from a fresh clone.

Bot / agent-driven QA

Per §11.4.83 operative rule (4): bot-driven / agent-driven QA automation **MUST** preserve the **full conversation thread** as the proof artefact. A bot that runs the round-trip but stores only the final PASS/FAIL line is itself a §107 bluff at the QA-automation layer. The bot writes the same `transcript.md` shape a human would — both halves of every exchange.

Directory layout

```
docs/qa/
├── README.md           # this file (the convention)
├── <run-id>/          # one directory per feature run
│   ├── transcript.md   # full bidirectional thread (required)
│   └── <materials...>  # screenshots / payloads / audio (as produced)
└── ...
```

Enforcement gate

`scripts/verify_qa_evidence.sh` scans feature-shipping commits (those touching non-test code under `helix_code/internal/**`, `helix_code/cmd/**`, or `helix_code/applications/**`, excluding `*_test.go`) and reports when a feature commit has no matching `docs/qa/<run-id>/` directory. It has two modes:

- **Advisory (default)** — `scripts/verify_qa_evidence.sh [N]`. Scans the last N commits (default 20), **WARNs**, and **ALWAYS** exits 0. For ad-hoc visibility; not wired into any git hook.
- **Enforcing** — `scripts/verify_qa_evidence.sh --enforce --since <ref>`. Exits **1** if any in-scope feature-shipping commit lacks its `docs/qa/<run-id>/` directory, **0** when clean. This is the §11.4.83 operative-rule-(5) release gate. The operator **authorised** promotion to a blocking release gate on **2026-05-28 (HXC-019)**.

Baseline scoping (`--since` is mandatory in `--enforce`)

The `docs/qa/` convention did not exist before it was introduced, so commits predating it cannot be expected to carry evidence. The baseline is the commit that **added** `docs/qa/README.md` — `ed84f90e` (2026-05-28). Find it with:

```
git log --diff-filter=A --format='%H %cI %s' -- docs/qa/README.md
```

`--enforce` evaluates only the range `<baseline>..HEAD` (descendants of the baseline, by merge-ancestry — not author-date sorting), so all pre-convention legacy history is exempt. `--since` is **mandatory** in enforcing mode; running `--enforce` without it is a misuse error (exit 2) — enforcing over the whole history would block on thousands of legacy commits and make HEAD un-releasable.

Baseline-bump history

The baseline is a moving historical line, advanced (never weakened) when already-pushed feature commits accumulated without their transcripts. A bump exempts the historical cohort as documented technical debt while the gate keeps **enforcing for every commit after the new baseline** — it moves the line forward only, it never erases the §11.4.83 requirement and never disables forward enforcement.

Date	Baseline SHA	Rationale
2026-05-28	ed84f90e7471fb683f7779bac80cdfd169620159	convention established (commit that added docs/qa/README.md, HXC-019)
2026-06-22	925169c98945ca0fee1e84dae53ad494e4897832	G7 remediation — 118 pre-existing, already-pushed feature commits in ed84f90e...HEAD had landed without a docs/qa/<run-id>/transcript. Those transcripts cannot be honestly retro-captured: the end-to-end runtime evidence §11.4.83 demands never existed for those commits, and fabricating 118 after-the-fact transcripts would itself be a §11.4 / §11.4.6 PASS-bluff. The honest remediation is to exempt the 118 as historical debt and keep the gate enforcing going forward, so the baseline is bumped to the 2026-06-22 HEAD (chore(submodule): bump constitution pointer to b8e73d8).

The baseline is encoded in two places, kept in sync: the hardcoded DEFAULT_BASELINE in scripts/gates/qa_evidence_gate.sh (the release-gate wrapper) and the qa_baseline default in the G7 gate of scripts/verify-all-constitution-rules.sh. The QA_EVIDENCE_BASELINE env var overrides both for a future re-baseline.

Per-commit opt-out: [no-qa-evidence]

A commit whose message (subject **or** body) contains the literal token [no-qa-evidence] is **exempt** from the gate. Use it for a change that trips the feature-shipping heuristic but ships no user-facing feature — a pure refactor, a governance-only edit, or a doc-only change. The token is

the single documented escape hatch; an untagged feature commit with no docs/qa/<run-id>/ directory always fails the enforcing gate.

Release-gate wiring (release-gate ONLY)

scripts/release-gate-test.sh invokes scripts/gates/qa_evidence_gate.sh, which runs the scanner with --enforce --since <baseline> and makes the whole release gate FAIL on any violation. Per the §11.4.83 mandate wording (“release gates”), the enforcing gate is deliberately **NOT** wired into pre-commit / pre-push git hooks — only into the release gate.

The paired-mutation meta-test scripts/tests/verify_qa_evidence_meta_test.sh (§1.1 anti-bluff) proves the gate fails when evidence is missing and passes when present.

See docs/scripts/verify_qa_evidence.md for the companion guide.

Worked example

docs/qa/HXC-016/ is a seed worked example: it records the §11.4.69–97 governance-cascade verification (HXC-016, closed Completed (→ Fixed.md) in docs/Fixed.md) in the transcript format above, demonstrating the full bidirectional thread + captured command output for a real, already-completed feature.